Разбор и критика решения Factmap

Дата разбора: 2026-10-27. Ветка `feat/hotel-agent-mvp`, коммит `4db0a14`. Стенд: образы `hotel-agent-backend:hotel-mvp-20261027` и `hotel-agent-frontend:hotel-mvp-20261027`, база `imresamu/postgis:16-3.4`, провайдер модели `openai_compat`. Требования, с которыми сверялось решение: `brief/TASK.md` и `brief/requirements.md` (R1–R9, O1–O5).

Как проверялось

Способ	Что именно
Гейты	`ruff`, `mypy`, `pytest tests/unit`, `pytest tests/contract` в `backend/`; `npm run verify` в `frontend/`
Живые запросы	Два запроса в `POST /api/v1/chat/messages` на поднятом стенде, RU и EN, с замером времени и разбором ответа целиком
База	Прямые запросы в PostGIS: объёмы, покрытие, провенанс, свежесть фактов
Код	Композиционный корень, сценарий поиска, агентный цикл пожелания, оценка совпадения, ранжирование, чат, фронтовый показ фактов

Чего в разборе нет: интеграционных тестов (нужна отдельная база), e2e, визуальной проверки в браузере, замеров под нагрузкой и прогона 50 запросов по виду из окна.

Как решение работает

Сообщение из чата попадает в `Chat.send`. Диалоговый слой решает, это консультация или поиск: консультация отвечает текстом, поиск уходит в `SearchHotels`.

Дальше поиск идёт по шагам: модель разбирает запрос на пожелания, радиусы и ограничения (`parse_query`); по подсказке места, семантике и названным объектам набираются кандидаты; для каждого пожелания запускается агентный цикл `WishLoop` — до 32 шагов и 180 секунд, с журналом вызовов, бюджетом на инструмент, отловом повторов и застоя; инструменты считают факты по PostGIS, рельефу Copernicus GLO-30, пешему графу OSM и линии видимости, читают веб-страницы и афишу; результат пожелания проходит проверку критериев и требование цитаты, иначе остаётся «неизвестно». Затем `score` складывает совпадения в оценку, `RecommendationReview` отбрасывает неподходящие рекомендации, ответ собирается текстом и фактами, геометрия доказательств сохраняется и уходит на карту.

Данные на стенде: 34 087 отелей, 27 488 779 контуров зданий, 12 544 факта, 269 926 геометрий доказательств, 89 категорий номеров, 840 сообщений чата.

Что сделано хорошо

1. **Значение факта архитектурно не может прийти от модели.** У `LLMPort` нет метода, который вернул бы значение характеристики; модель разбирает запрос, планирует инструменты и формулирует вопросы. Это прямо отвечает требованию ТЗ «агент, который просит LLM угадать, — не решение».

2. **Агентный цикл не даёт модели объявить совпадение без доказательства.** `WishLoop` проверяет критерии, требует цитату источника, возвращает модель назад при необоснованном `finish`, ловит повторяющиеся вызовы и застой, держит бюджет на инструмент и срок. Это редкая и ценная часть.
3. **Полный провенанс в интерфейсе.** Карточка факта показывает источник, уверенность, дату расчёта, дату наблюдения, срок годности и историю значений.
4. **Разрешение противоречий написано как доменная служба** с тирами `hotelier > fresh_computed > osm_tag > stale_computed`, с состоянием «нерешено», когда уверенности близки, и с записью в таблицу `conflicts`.
5. **Честные формулировки вместо выдуманных значений.** «Неизвестно» не превращается в «нет»; отдельно сказано, что остановка проверки не доказывает отсутствие свойства.
6. **Глубина по заявленному региону реальна.** В границах Сочи 854 отеля, факты есть у 848 — 99,3 %.
7. **Гейты зелёные.** Бэкенд: `ruff` без замечаний, `mypy` на 555 файлах, 984 юнит-теста, 37 контрактных. Фронтенд: 36 проверок правил, 103 файла тестов, 560 тестов, сборка проходит.
8. **Документация признаёт границы.** `docs/limitations.md` и `docs/validation-plan.md` сами пишут, что точность не измерена и эталонной разметки нет.

Критика

P0. Ранжирование вырождено: порядок выдачи фактически алфавитный

Живой запрос «тихий отель в Сочи с видом на закат недалеко от моря» вернул 20 отелей, **у всех 20 оценка 0.365**, порядок: Abris, AIDA, Ales, Alpha Sirius, Azimut, Black Sea, Bliss House, Bridge Family Resort. Второй запрос, на английском, — те же 20 отелей, **у всех 0.547**, тот же алфавитный порядок.

Причина цепочкой:

- `application/agent/wish_evidence.py:57` сводит измерение к логическому значению: и полное, и частичное совпадение дают `value = True`, величина уходит в `qualifiers`;
- `domain/services/scoring.py:21` считает вклад как `вес × ведро × уверенность`, где ведро — одно из трёх дискретных значений;
- `application/search_ranking.py:24` замыкает сортировку на `hotel.name.casefold()`.

Проверка по базе: у отеля `n10058556917` шум 0.0, у `n10118233110` — 3.8 при пороге 3.0, и оба получают `value = true`, уверенность 0.6, то есть **одинаковый вклад в оценку**. Отель на тихой улице и отель у дороги стоят в списке рядом, а их взаимный порядок задан первой буквой названия.

Для жюри это читается как «ранжирования нет». Чинится без переделки: нормировать величину относительно порога и включить её в оценку монотонно, либо хотя бы добавить `native_value` в ключ сортировки до имени.

P0. Ни одного вопроса отельеру и ни одного конфликта на живом стенде

`hotelier_questions` — 0 строк, `conflicts` — 0 строк при 12 544 фактах. Обязательные требования R2 и R3 закрыты кодом, юнит-тестами и контрактными тестами, но **на стенде, который увидит жюри, ни одного вопроса и ни одного разобранного противоречия не существует**. Цель `make plan-questions` есть в Makefile и на этом стенде не запускалась.

Это самый дешёвый и самый дорогой по последствиям пробел: два из пяти обязательных пунктов ТЗ проверяемы только чтением кода.

P0. Провенанс наполовину пустой

- `source_url` заполнен у **0 фактов из 12 544**;
- `evidence` непустой у **1 факта из 12 544**.

Колонки добавлены миграцией `20261026200000_fact_provenance_details` и не наполняются. У фактов из веб-источников ссылки на источник в базе нет, хотя ТЗ требует «источник» у каждого значения, а интерфейс показывает поле источника. Для расчётных фактов `source_type = computed` и `method` осмысленны, и это спасает картину, но web-путь остаётся без адреса источника.

P0. 1 453 факта просрочены, перепроверки нет

Фактов с истёкшим `valid_until` — 1 453 из 12 544 (11,6 %). Срок годности заполняется у всех фактов, но ни одна команда не перечитывает просроченные значения. Просроченный факт понижается только внутри разрешения конфликта; в выдачу он попадает как обычный. Это и есть дополнительная задача O4, и она не сделана.

P0. Вид на закат по-прежнему не вычисляется

Живой ответ стенда на главный демо-запрос: «Проверка не завершена: с видом на закат», факт — `unknown` с объяснением «Источники не дали ничего нового по этому пожеланию». То есть первый пример ТЗ и собственный демо-запрос продукта на стенде не работают, хотя `solar_position` и геометрия видимости готовы. Подробности и план — в `work/task-coverage/plan.md`.

P0. Демо-стенд и чистый клон — это разные продукты

На стенде 34 087 отелей и 27,5 млн зданий по всей России. Эти данные приходят из `make ingest-russia` и снапшота OSM на десятки гигабайт, которого в репозитории нет. Жюри из чистого клона получает **только Сочи**: 854 отеля с фактами у 848. Формально требование R8 (демо-данные в репозитории, без многочасовой загрузки) выполняется именно потому, что в клоне лежит только срез Сочи.

Опасность в несовпадении: если на защите показать запрос по Казани или Байкалу, а жюри повторит его из клона, результат будет другим. Нужно либо заявить Сочи как регион и показывать его, либо приложить воспроизводимый национальный срез.

P1. Латентность запроса: 78 и 43 секунды

Замеры живых запросов: 78,0 с для трёх пожеланий, 43,2 с для двух. Это не случайный всплеск, а устройство цикла: на каждое пожелание до 32 шагов, таймаут 180 с, и почти каждый шаг — это вызов модели. Поток прогресса по SSE смягчает ожидание, но не устраняет его.

P1. Поле `took_ms` в ответе чата всегда 0

`main.py` передаёт в сериализатор литеральный `0`: `search_schema(outcome, providers.registry, 0)`. Запрос, который шёл 78 секунд, в ответе сообщает `took_ms: 0`. Поле контракта, которое не просто неточно, а неверно — а чат единственный путь, которым пользуется интерфейс.

P1. Два параллельных реестра характеристик

В `characteristics/` девять описанных характеристик. В базе 292 различных ключа, из них 131 — `derived:<хеш>`, созданные агентом из текста пожелания (`wish_evidence.py:15`). У такого ключа нет описания, типа значения, единиц и метода; человеку он виден только потому, что интерфейс подставляет исходный текст пожелания вместо названия.

Отсюда два следствия. Первое: расширяемость (R4) на деле держится не на YAML-реестре, а на хешах, которые нигде не описаны. Второе: утверждение README «условие с характеристикой не из реестра отбрасывается» верно только для офлайн-разбора, а для живого пути сосуществует с 131 нереестровым ключом. Формулировку надо уточнить, иначе на вопрос жюри ответ будет выглядеть неправдой.

P1. Русский текст в прикладном слое: английская версия показывает русские объяснения

В `application/` и `domain/` 209 строк с русским текстом в 36 файлах. Живой запрос с `locale=en` вернул английский ответ ассистента и **русские объяснения фактов**: «Оценка 0–10 по расстоянию до дорог, железной дороги и баров OSM; не замер звука». Собственные правила проекта требуют двух языков интерфейса; фронтенд их соблюдает, бэкенд — нет.

P1. «Тихо» не может стать полным совпадением

Шум считается по сторонам здания, поэтому факт несёт ограничивающий признак, и оценка принудительно понижается до частичной (`domain/entities.py:70`, `scoring.py:60`). Логика честная: тихая сторона есть, но не весь отель. Последствие в том, что оценка никогда не достигает единицы, и ответ всегда содержит «ни у одного отеля не подтверждены все условия» — на обоих живых запросах именно так. Пользователь читает это как «ничего не нашли». Нужна формулировка, которая отличает «данных нет» от «подтверждено для западной стороны».

P1. Офлайн-гарантия верна только для Сочи

Вне границ локального DEM рельеф тянется из сторонних тайлов `tiles.mapterhorn.com`, дополнительная геометрия — из зеркала Overpass `overpass.private.coffee`, эмбеддинги — из `polza.ai`, веб-исследование — из открытого интернета. Фолбэков у этих источников нет, у двух из них нет и договорённостей об использовании. Утверждение «работает без ключей и сети» нужно читать как «в Сочи и по предрассчитанным фактам».

P1. Семантический индекс живёт вне репозитория

Индекс документов читается из `data/raw/semantic`, а `data/raw/` в `.gitignore`. В чистом клоне семантический поиск деградирует до лексического, и это нигде не видно пользователю. Плюс его построение требует платного ключа.

P2. Остальное

Что	Где	Почему это важно
`assert` в рабочем пути	`use_cases/chat.py`	С флагом `-O` проверки исчезают, ошибка станет `AttributeError` вместо внятной
Композиция разрослась	`main.py`, `create_app` ~150 строк; `DataExplorer` принимает 15 зависимостей	Добавление инструмента требует правки одного большого места
7 файлов длиннее 250 строк	`web_research.py` 296, `cli.py` 288, `data_tools.py` 279, `wish_loop.py` 262, `place_names.py` 254, `structured.py` 253, `search.py` 252	Собственное правило проекта нарушено (сгенерированный `schema.gen.ts` исключён законно)
Чанк карты 1,1 МБ	`pmtiles-protocol` в сборке фронтенда	Первая загрузка карты на слабой сети
Устаревший идентификатор модели	`.env.example` : `claude-sonnet-4-6`	Актуальны модели Claude 5 (`claude-opus-5-5` , `claude-sonnet-5-5`); жюри может проверить
Нет входа и прав	весь API	Знающий `conversation_id` читает, переименовывает и удаляет переписку; отельер правит факты без входа. Задокументировано, но на стенде это открытый API
`docs/requirements-map.md` устарел	строки про переданное ТЗ и про `/compare` 404	Жюри читает именно этот файл

Числа прогона

Что	Значение
Живой запрос RU, три пожелания	78,0 с, 20 отелей, все с оценкой 0.365
Живой запрос EN, два пожелания	43,2 с, 20 отелей, все с оценкой 0.547
Отели / здания / факты / геометрии	34 087 / 27 488 779 / 12 544 / 269 926
Отели с хотя бы одним фактом	2 632 из 34 087 по России (7,7 %); 848 из 854 в Сочи (99,3 %)
Ключи характеристик	292, из них 131 `derived:<хеш>`, 9 описаны в `characteristics/`
Провенанс	`source_url` у 0 фактов, `evidence` у 1 факта, средняя уверенность 0.621
Свежесть	1 453 факта просрочены, у всех фактов срок годности заполнен
Вопросы отельеру / конфликты	0 / 0
Категории номеров	89
Гейты	бэкенд: ruff чисто, mypy 555 файлов, 984 юнит-теста, 37 контрактных; фронтенд: 36 проверок, 560 тестов, сборка

Что делать в первую очередь

1. Включить величину измерения в оценку и в сортировку — иначе выдача выглядит алфавитной.
2. Поставить на стенд хотя бы один вопрос отельеру и один разобранный конфликт, чтобы R2 и R3 можно было увидеть, а не прочитать в коде.
3. Заполнить `source_url` и `evidence` для фактов из веба.
4. Сделать перепроверку просроченных фактов (O4) и показать её на одном примере.
5. Досчитать вид на закат.
6. Согласовать, что показываем: Сочи из клона или Россию со стенда, и сказать это прямо.

Источники чисел: живые ответы стенда и прямые запросы к базе 2026-10-27; гейты запущены в этот же день на коммите `4db0a14`.